

agent_skills — Usage Manual

A portable coding-agent harness for documented workflows

Generated 2026-05-22

agent_skills lets an LLM coding agent (Codex, Claude Code, Cursor, Continue, and similar tools) drive documented workflows: route a project to the right workflow, validate inputs, build commands *only* from declared templates, parse outputs, run sanity checks, and write audit logs and memory — **without inventing commands, parameters, tools, or databases**. It is not a chatbot demo: the hooks and the eval runner are plain Python you can call directly, useful even with no LLM. The local repository is always the source of truth.

How it is organised

The pack is a portable harness with a thin project layer on top, so the engine can be lifted into any repository:

core/	adapters/claude_code/	project/
Project-agnostic engine: hooks (preflight, command_builder, audit, generic parsers/validation), the skill schema, the agnostic AGENTS.md contract, the memory format spec, and the offline runner.	The only Claude-specific part: CLAUDE.md operating rules, an install guide, and a sample .claude/settings.json hook wiring.	This repository's content: tool-specific parsers and threshold validators, the routing prompt, and the benchmark tasks/fixtures.

The 13 skill YAMLs in skills/ are the optional project content the harness drives. The legacy hooks/ facade and evals/run_benchmarks.py are kept so existing imports keep working; new code imports from agent_skills.core.hooks (portable) or agent_skills.project.hooks (project-specific).

Quick start

Requirements: Python 3 and PyYAML (already in the project's environment.yaml). No bioinformatics tools are needed to run the offline checks.

```
git clone https://github.com/ttmgr/GenomicsForOneHealth.git
cd GenomicsForOneHealth

# run the offline check suite (no LLM, no external tools)
python agent_skills/evals/run_benchmarks.py

# inspect a skill from Python
python -c "from agent_skills.hooks import command_builder as cb; \
    print(cb.load_skill_yaml('agent_skills/skills/\
    cre_plasmid_clustering.yaml')['skill']['display_name'])"
```

A green run prints `PASS 18 FAIL 0 SKIP 4` (the four skips are routing tasks a human reviews).

Driving a workflow with a coding agent

Point your agent at the `agent_skills/` directory. It reads the pack README, then `core/AGENTS.md` (the agnostic contract; Claude Code uses `adapters/claude_code/CLAUDE.md`), then the relevant skill. The procedure (from `project/prompts/use_skill_pack.md`):

- 1 **Route** the request to a single skill by matching molecule/sample type and goal against each skill's domain, description, and supported inputs. If nothing fits, say so.
- 2 **Load** the skill with `command_builder.load_skill_yaml`.
- 3 **Preflight**: run the skill's pre-hooks (`core/hooks/preflight.py`) against the real inputs, databases, and tools; stop if a check fails.
- 4 **Validate parameters** with `validate_required_parameters`. Ask for any missing required parameter; never fill an unprovided value, never accept an undeclared one.
- 5 **Build** commands with `build_commands_for_skill` and present them. Commands are built, not run; destructive ones need explicit confirmation.
- 6 **Parse** outputs with the declared post-hooks (generic in `core/hooks/parsers.py`, tool-specific in `project/hooks/parsers_genomics.py`).
- 7 **Validate outputs**: run the validation hooks and report every raised flag with its severity.
- 8 **Audit & remember**: write a JSON audit log, then distil the run into the memory store with `audit.append_run_record('agent_skills/memory', ...)`.
- 9 **Report**: what was run, key parsed metrics, validation flags, and explicit caveats.

The rules that never bend

- **The local repository is the source of truth.** Use only commands, parameters, tools, and databases the selected skill declares.
- **Never invent.** If a command is not documented, do not synthesise one — surface the skill's `needs_review` notes instead.

- **Build, never execute.** The hooks return command strings; a human runs them.
- **External references are for comparison only** and never override a local command.
- **All outputs are suggestions**, not biological, clinical, regulatory, or diagnostic conclusions — say so in the report.

Memory

Durable memory persists what code and git history don't: architectural decisions, distilled run summaries, and operator preferences. The format (in `core/memory/README.md`) is one fact per file with frontmatter, a single `MEMORY.md` index, and `[[wikilinks]]` — mirroring the Claude Code memory convention, so the adapter maps onto it with no translation. The live store for this repo is `agent_skills/memory/`; `audit.append_run_record` writes `type: run` entries automatically.

Adding a new skill

Follow `core/prompts/extract_new_skill.md`: read the workflow's docs and scripts in full; extract tools (with version pins), every command verbatim, I/O types, and databases; write a YAML conforming to `core/schemas/skill.schema.json`; set `source_files` for traceability; flag anything ambiguous in `needs_review`; add a benchmark task. Then check it against `core/prompts/validate_skill.md` and re-run the offline suite.

Installing the harness in another project

The core has no dependency on this repository's content, so lifting it is a copy plus three small wirings (see `adapters/claude_code/README.md`):

- 1 Copy `agent_skills/core/` and `agent_skills/adapters/` into the target repo.
- 2 Add that repo's own skill YAMLs under `skills/` and any project-specific parsers/validators under `project/hooks/` (the core works with an empty project layer too).
- 3 Create a memory directory seeded with a `MEMORY.md` index and point `append_run_record` at it; merge `settings.hooks.sample.json` into `.claude/settings.json`.

Caveats

All model outputs from this pack are suggestions, not biological, clinical, regulatory, or diagnostic conclusions. Commands are built, not executed; the human remains responsible for running them and interpreting the results.